

سیستم‌های نهفته

استاد انصاری

عاطفه قندهاری - مبینا حیدری - نیکا قادری
زمستان ۱۴۰۴



گزارش پروژه

سیستم امنیتی هوشمند

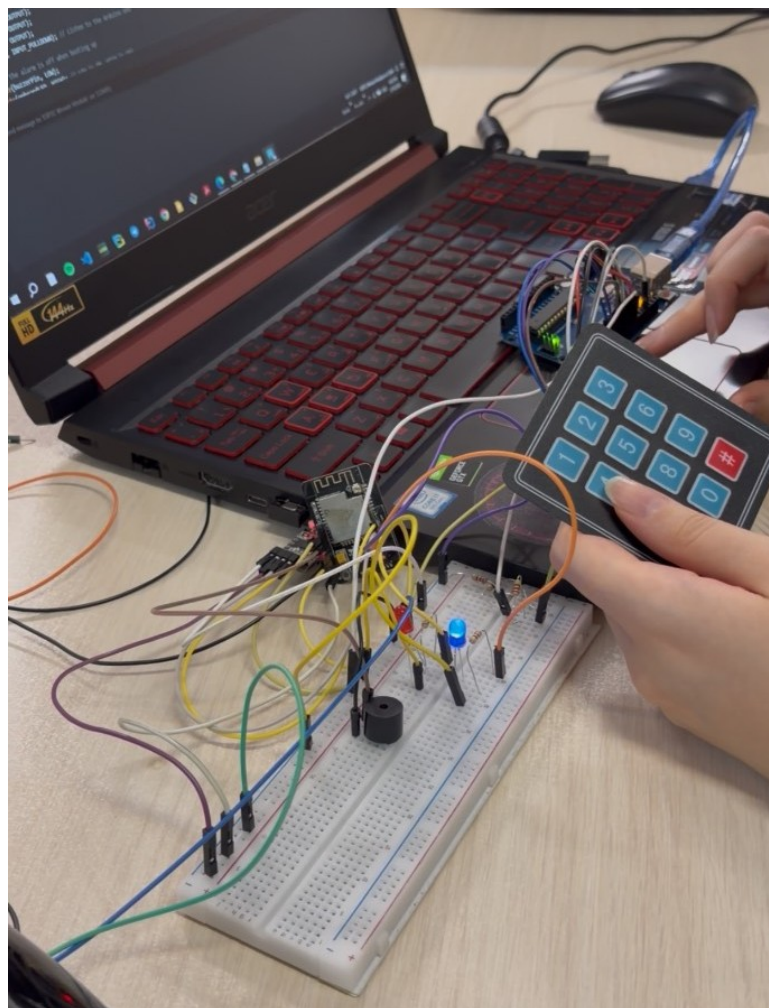
تاریخ گزارش: ۳ اسفند ۱۴۰۴

فهرست مطالب

۱	۱	بررسی اجمالی فاز پیاده‌سازی و یکپارچه‌سازی
۳	۲	توسعه معماری سخت‌افزار: سیستم دو مغزی و تطبیق ولتاژ
۳	۱.۲	بحران کمبود پین و معرفی معماری دو مغزی (Two-Brain System)
۳	۲.۲	طراحی مقسم ولتاژ (Voltage Divider) برای ارتباط ایمن
۴	۳	عیب‌یابی چالش‌های سخت‌افزاری و سیستمی
۴	۱.۳	تداخلات جریان راه‌اندازی (Brownout) و ماژول FTDI
۴	۲.۳	تداخل حافظه PSRAM و خطای LoadProhibited
۵	۳.۳	کالیبراسیون و پایداری‌سازی سنسور PIR
۵	۴	یکپارچه‌سازی نرم‌افزار، الگوریتم‌ها و شبکه
۵	۱.۴	الگوریتم عکس‌برداری Single-Shot
۶	۲.۴	دور زدن فیلترینگ تلگرام با معماری ابری (Cloudflare Worker)
۶	۵	نتیجه‌گیری فاز دوم و جمع‌بندی
۷	۶	پیوست ۱: کدهای نهایی (Source Code)
۷	۱.۶	کد کنترل پین دسترسی (Arduino Uno)
۹	۲.۶	کد هسته مرکزی، سنسورها و شبکه (ESP32-CAM)
۱۵	۷	منابع

۱ بررسی اجمالی فاز پیاده‌سازی و یکپارچه‌سازی

هدف از اجرای فاز دوم این پروژه، انتقال از فاز طراحی و شبیه‌سازی به پیاده‌سازی عملی، مونتاژ سخت‌افزاری، توسعه نرم‌افزار نهایی و غلبه بر چالش‌های پیش‌بینی‌نشده فیزیکی و شبکه‌ای برای ساخت سیستم (ESP32-CAM + Arduino Uno) Smart Security System بود. در این مرحله، قطعات سخت‌افزاری شامل برد پردازشی اصلی (ESP32-CAM)، سنسور تشخیص حرکت (PIR)، هشدارهای محلی (آژیر و LEDهای وضعیت) و ماژول رابط کاربری کیبورد ماتریسی (HX-543) به صورت فیزیکی پیکربندی و برنامه‌نویسی شدند. روند پیاده‌سازی نشان داد که طراحی اولیه با محدودیت‌های ذاتی سخت‌افزار، به ویژه در زمینه کمبود پین‌های ورودی/خروجی (GPIO) در برد ESP32 و همچنین تفاوت‌های سطح منطقی ولتاژ قطعات، مواجهه است. در نتیجه، معماری سیستم به یک معماری «دو مغزی» (Two-Brain System) ارتقا یافت که در آن برد Arduino Uno به عنوان پردازنده کمکی برای مدیریت رابط کاربری اضافه گردید. علاوه بر چالش‌های سخت‌افزاری مانند نویزهای الکترومغناطیسی بردبرد، افت ولتاژ ناگهانی در لحظه روشن شدن وای‌فای (Brownout) و تداخلات پین‌های مربوط به حافظه داخلی (PSRAM)، یکی از بزرگترین چالش‌های این فاز، محدودیت‌های دسترسی به اینترنت (فیلترینگ) در اتصال به سرورهای تلگرام بود. در این گزارش، مراحل دقیق عیب‌یابی، کالیبراسیون و راهکارهای نرم‌افزاری نوآورانه‌ای که برای دور زدن این فیلترینگ با استفاده از واسطه‌های ابری (Cloudflare) پیاده‌سازی شد، به تفصیل و مرحله‌به‌مرحله شرح داده شده است. سیستم نهایی اکنون قادر است با پردازش رمز عبور، محیط را مانیتور کرده و ضمن ایجاد بازدارندگی صوتی و بصری فوری، شواهد تصویری را با موفقیت و پایداری بالا به صورت Real-time به تلفن همراه کاربر ارسال نماید.



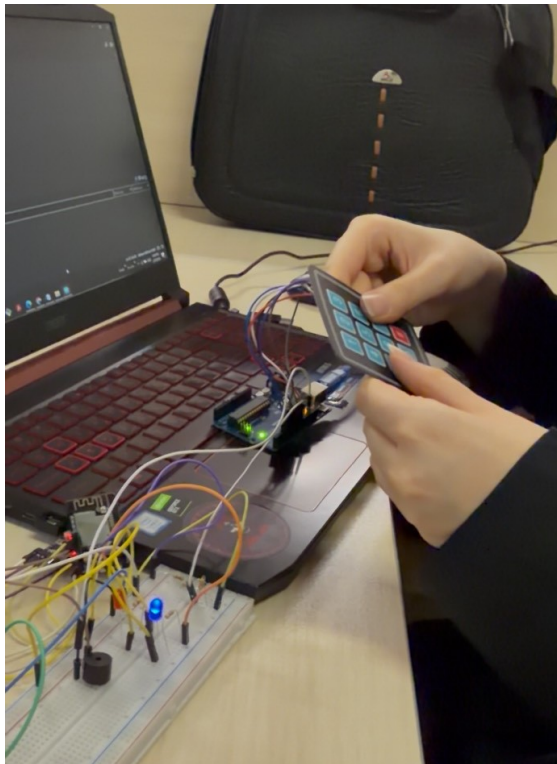
شکل ۱: نمای کلی مدار یکپارچه‌شده در فاز دوم، شامل هر دو پردازنده (ESP32) و (Arduino)، کیپد، و هشدارهای محلی

۲ توسعه معماری سخت افزار: سیستم دو مغزی و تطبیق ولتاژ

۱.۲ بحران کمبود پین و معرفی معماری دو مغزی (Two-Brain System)

بر اساس الزامات فاز اول، سیستم باید دارای یک پین کنترل دسترسی برای فعال و غیرفعال کردن دزدگیر از طریق رمز عبور می‌بود. قطعه در نظر گرفته شده برای این کار، یک کپید ماتریسی ۴ در ۳ مدل (HX-543) بود که برای اتصال به پردازنده به ۷ پین ورودی دیجیتال نیاز دارد. در حین پیاده‌سازی مشخص شد که برد ESP32-CAM تقریباً تمام پین‌های GPIO خود را به صورت داخلی برای ارتباط با لنز دوربین (OV2640) و درگاه کارت حافظه MicroSD مصرف کرده است و عملاً پین خالی کافی برای اتصال کپید در دسترس نیست.

برای حل این چالش حیاتی، تصمیم گرفته شد تا از یک برد Arduino Uno به عنوان پردازنده ثانویه (مغز دوم) استفاده شود. در این معماری جدید، آردوینو به عنوان یک «پین کنترل دسترسی» مستقل عمل می‌کند. ۷ پین کپید به پین‌های دیجیتال ۹ تا ۳ در آردوینو متصل شدند. وظیفه این برد، خواندن کلیدهای فشرده‌شده، بررسی تطابق آن‌ها با رمز عبور تعیین شده و در نهایت مدیریت وضعیت سیستم است. پس از تأیید رمز عبور، آردوینو یک سیگنال دیجیتال (HIGH برای فعال و LOW برای غیرفعال) از طریق پین ۱۰ خود به سمت پین ۲ GPIO در برد ESP32-CAM ارسال می‌کند تا دوربین را در جریان وضعیت امنیتی قرار دهد.



شکل ۲: برد Arduino Uno که به عنوان پردازنده رابط کاربری به مدار اضافه شد
شکل ۳: تست عملکرد کپید ماتریسی و اعتبارسنجی رمز عبور توسط آردوینو

۲.۲ طراحی مقسم ولتاژ (Voltage Divider) برای ارتباط ایمن

اضافه شدن آردوینو به مدار، یک خطر سخت‌افزاری جدی به همراه داشت: تفاوت سطح منطق ولتاژ (Logic Level). برد آردوینو با منطق ۵ ولت کار می‌کند؛ به این معنا که سیگنال HIGH خروجی آن معادل ۵ ولت کامل است. از سوی دیگر، تراشه مرکزی ESP32 مستقیماً با منطق ۳.۳ ولت کار می‌کند. اگرچه ماژول دوربین دارای یک رگولاتور داخلی برای تبدیل ۵ ولت منبع تغذیه به ۳.۳ ولت است، اما پین‌های دیتای آن (GPIOها) این رگولاتور را دور می‌زنند و اتصال مستقیم سیگنال ۵ ولتی آردوینو به پین دیتا، منجر به سوختن قطعی تراشه ESP32 می‌شد.

برای حل این مشکل بدون نیاز به ماژول‌های مبدل لاجیک اضافی، یک مدار مقسم ولتاژ با استفاده از قطعات موجود طراحی شد. با توجه به در دسترس بودن مقاومت‌های ۲۲۰ اهمی، ساختار زیر برای کاهش ایمن ولتاژ به کار گرفته شد: سه عدد مقاومت ۲۲۰ اهمی به شکل یک شبکه «T» متصل شدند. مقاومت اول در مسیر سیگنال خروجی از آردوینو (پین ۱۰) به عنوان پل ارتباطی قرار گرفت. سپس از محل اتصال این مقاومت به پین ۲ GPIO دوربین، دو مقاومت ۲۲۰ اهمی دیگر به صورت سری به زمین (GND) متصل شدند. از نظر ریاضی این ترکیب دقیقاً ولتاژ ۵ ولت را

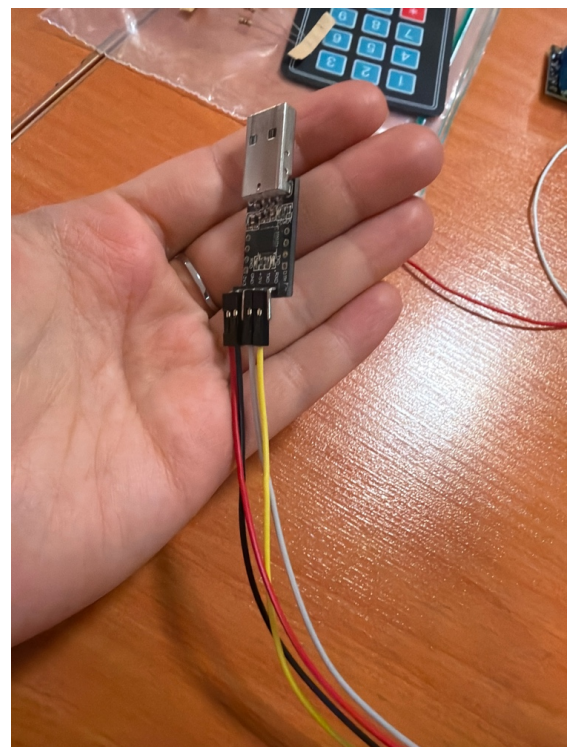
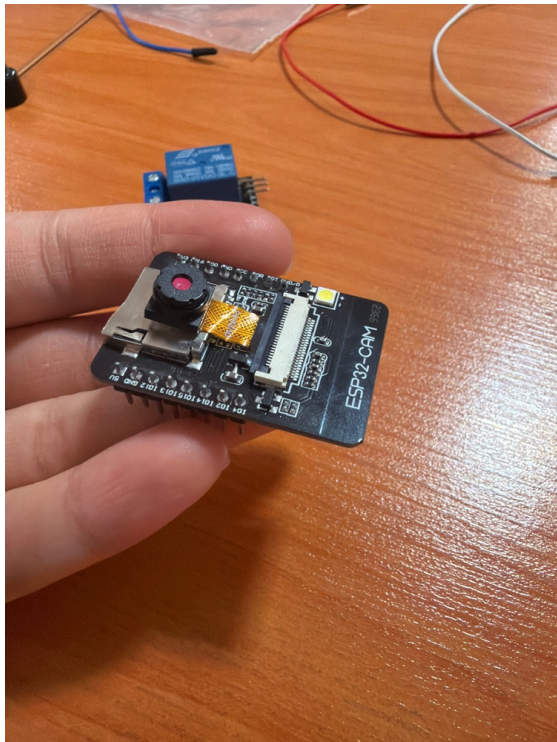
به ۳۳.۳ ولت کاهش داده و ارتباط داده‌ای کاملاً ایمن و بدون نویز را بین دو پردازنده برقرار کرد. همچنین برای هم‌مرجع شدن سیگنال‌ها، پین‌های GND هر دو برد به یکدیگر متصل شدند.

۳ عیب‌یابی چالش‌های سخت‌افزاری و سیستمی

۱.۳ تداخلات جریان راه‌اندازی (Brownout) و ماژول FTDI

یکی از مشکلات کلافه‌کننده در مراحل اولیه، ری‌استارت شدن مداوم برد در لحظه فراخوانی تابع اتصال وای‌فای بود. در مانیتور سریال، خطای Brownout detector was triggered مشاهده می‌شد. دلیل این امر آن بود که روشن شدن آنتن وای‌فای به صورت لحظه‌ای نیازمند جریانی بالغ بر ۵۰۰ میلی‌آمپر است. کابل‌ها و درگاه‌های USB کامپیوتر که از طریق ماژول پروگرامر FTDI برق مدار را تأمین می‌کردند، توانایی ارائه این جریان ناگهانی را نداشتند؛ در نتیجه ولتاژ افت کرده و سیستم برای محافظت از خود خاموش می‌شد.

برای رفع این مشکل، پروسه‌ی «آپلود» کد از پروسه‌ی «اجرا» به طور کامل تفکیک شد. برد FTDI صرفاً با اتصال مستقیم جابجایی به پین‌های سریال و GPIO 0 (برای ورود به حالت فلش) جهت پروگرام کردن تراشه استفاده شد. پس از اتمام برنامه‌نویسی، مدار از FTDI جدا شده و برق آن به صورت مستقل و پایدارتر روی بردبورد تأمین گردید.



شکل ۵: هسته اصلی سیستم؛ برد توسعه ESP32-CAM

شکل ۴: ماژول برنامه‌نویس FTDI مورد استفاده برای فلش کردن تراشه مرکزی

۲.۳ تداخل حافظه PSRAM و خطای LoadProhibited

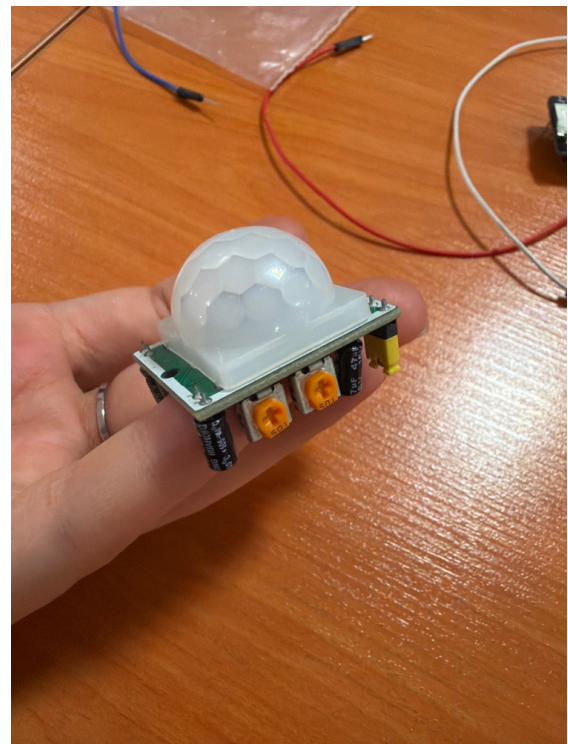
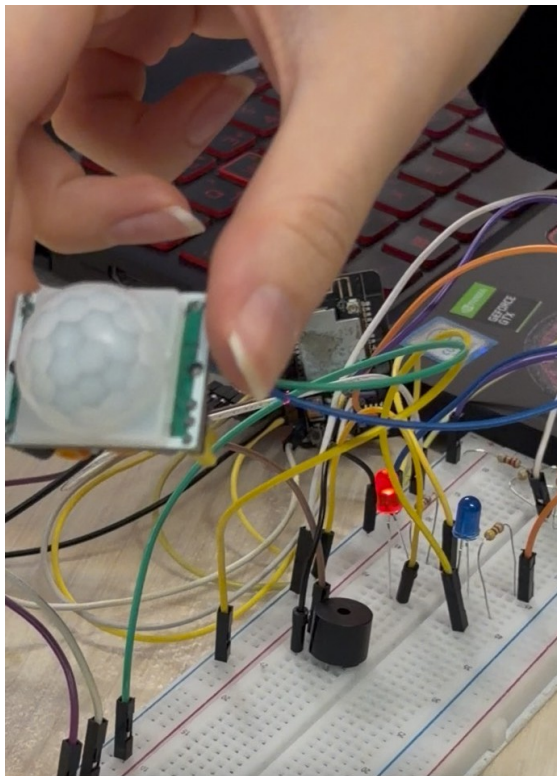
هنگام اضافه کردن LEDهای نشانگر وضعیت به مدار، اختصاص پین 16 GPIO به LED حالت عادی منجر به بروز خطای مهلک Guru Meditation Error: Core 0 panic'd (LoadProhibited) شد. پس از بررسی معماری داخلی ماژول AI-Thinker، مشخص شد که GPIO 16 به عنوان خط Chip Select برای تراشه حافظه جانبی (PSRAM) به صورت سخت‌افزاری رزرو شده است. زمانی که برنامه سعی می‌کرد پین ۱۶ را به عنوان خروجی تنظیم کند، ارتباط پردازنده با حافظه‌ای که برای پردازش تصویر و کش وای‌فای ضروری بود، قطع می‌شد. راه‌حل این چالش، بازطراحی تخصیص پین‌ها بود. پین مربوط به LED عادی به GPIO 4 منتقل شد. نکته جالب توجه این بود که GPIO 4 به طور داخلی به فلش لایت قدرتمند تعبیه شده روی خود ماژول نیز متصل است؛ در نتیجه، چراغ وضعیت عادی سیستم همزمان به عنوان یک نورپردازی قدرتمند محیطی عمل کرده و کاربرد امنیتی سیستم را ارتقا داد.

۳.۳ کالیبراسیون و پایداری سازی سنسور PIR

سنسور HC-SR501 در محیط آزمایشگاهی هشدارهای کاذب بی‌وقفه‌ای تولید می‌کرد. با ایزوله کردن سنسور زیر یک پوشش بسته، اثبات شد که مشکل ناشی از نویز محیطی نیست، بلکه به دلیل تنظیمات پیش‌فرض مازول و حساسیت الکتریکی پین‌هاست. اقدامات کالیبراسیون زیر برای پایداری سازی آن انجام شد:

۱. کالیبراسیون سخت‌افزاری: پتانسیومتر تنظیم زمان تا انتها در جهت پادساعتگرد چرخانده شد تا زمان فعال ماندن پین پس از تشخیص به حداقل مقدار ممکن (حدود ۳ ثانیه) کاهش یابد. جامپر روی سنسور نیز در حالت Single Trigger قرار گرفت تا از ارسال پالس‌های لوپ جلوگیری شود.

۲. تنظیم نرم‌افزاری: در کد ESP32، پین ورودی سنسور به جای INPUT ساده، به عنوان INPUT_PULLDOWN تعریف شد تا در زمان‌های عدم تشخیص، پین در حالت شناور و حساس به نویز قرار نگیرد و کاملاً به صفر منطقی متصل باشد.



شکل ۷: تست موفق هشدارهای محلی و فعال‌سازی آژیر و چراغ‌ها هنگام تشخیص

شکل ۶: سنسور تشخیص حرکت HC-SR501 و تنظیم پتانسیومترهای حساسیت

۴ یکپارچه‌سازی نرم‌افزار، الگوریتم‌ها و شبکه

۱.۴ الگوریتم عکس‌برداری Single-Shot

یکی از مشکلاتی که در منطق اولیه سیستم وجود داشت، ارسال متوالی ده‌ها عکس در صورت حضور پیوسته یک فرد در میدان دید سنسور بود که می‌توانست منجر به پر شدن حافظه گوشی کاربر و اسپم شدن پیام‌ها شود. برای رفع این مشکل، یک مفهوم ماشین حالت تحت عنوان فلگ که می‌توانست منجر به پر شدن حافظه گوشی کاربر و اسپم شدن پیام‌ها شود. با استفاده از این متغیر منطقی، هنگامی که حرکت برای اولین بار تشخیص داده می‌شود، سیستم بلافاصله هشدارهای صوتی و بصری را روشن کرده، تنها یک عکس ثبت می‌کند و سپس فلگ را روشن می‌کند. دوربین تا زمانی که فرد از محیط خارج نشده و سیگنال سنسور به حالت عادی برنگردد (که به معنای پایان رویداد امنیتی است)، در حالت قفل (Lockout) باقی می‌ماند، در حالی که آژیر و چراغ‌ها همچنان به کار خود ادامه می‌دهند.

۲.۴ دور زدن فیلترینگ تلگرام با معماری ابری (Cloudflare Worker)

بخش نهایی الزامات پروژه، ارسال تصویر سارق به یک ربات تلگرامی بود. در ابتدا با مراجعه به تلگرام و استفاده از ابزارهای BotFather و IDBot، توکن احراز هویت ربات و شناسه چت مقصد دریافت شد. برای جلوگیری از پر شدن حافظه مازول، کدهای ارسال تصویر از کتابخانه‌های آماده سنگین خارج شده و به صورت یک درخواست مستقیم HTTP POST با فرمت multipart/form-data طراحی شد تا عکس در قالب بسته‌های کوچک استریم شود.

با وجود اجرای بی نقص منطق داخلی، در زمان تست واقعی، سیستم هنگام ارتباط با سرورهای تلگرام دچار خطای مسدودیت شبکه (Failed to connect) می‌شد. علت این امر، اعمال محدودیت‌ها و فیلترینگ سراسری بر روی دامنه‌های تلگرام در اینترنت کشور است. از آنجا که میکروکنترلرها توانایی اجرای نرم‌افزارهای عبور از تحریم (VPN) را ندارند، یک راهکار ابری بسیار کارآمد به عنوان معماری جایگزین پیاده‌سازی شد: به جای ارتباط مستقیم، از پلتفرم توسعه Cloudflare Workers استفاده گردید. یک اسکریپت جاوا اسکریپت به عنوان Reverse Proxy در کلودفلر مستقر شد که وظیفه آن رهگیری درخواست‌های HTTP و تغییر دامنه‌ی هدف به دامنه‌ی تلگرام است. دامنه اختصاص داده شده توسط کلودفلر فیلتر نبوده و در دسترس سیستم قرار داشت. کد نوشته شده در ESP32-CAM به گونه‌ای اصلاح شد که به جای آدرس تلگرام، درخواست خود را به آدرس واسط کلودفلر ارسال کند. سرور کلودفلر در کسری از ثانیه تصویر را دریافت کرده و به سرور تلگرام رله (Forward) می‌کند. این مکانیزم با موفقیت کامل توانست محدودیت اینترنت را در یک سیستم تعبیه شده دور بزند.

۵ نتیجه گیری فاز دوم و جمع بندی

فاز دوم پروژه سیستم امنیتی هوشمند (ESP32-CAM + Arduino Uno) با دستیابی به تمامی اهداف تعیین شده به اتمام رسید. معماری ابتدایی تک‌پردازنده‌ای با اضافه کردن آردوینو برای رابط کاربری به یک معماری دوگانه و پایدار ارتقا یافت. خطرات ولتاژی از طریق مدارهای مقاومتی مهار شد، و چالش‌های پردازشی با شناخت دقیق سخت‌افزار (مانند ایزوله کردن پین‌های PSRAM) و پیاده‌سازی الگوریتم‌های مدیریت بافر، با موفقیت کنترل گردید. در نهایت، پیچیده‌ترین مانع محیطی پروژه، یعنی عدم دسترسی به شبکه آزاد، با ابتکار عمل در استفاده از واسط‌های ابری به طور کامل برطرف شد. این دستگاه اکنون یک پکیج امنیتی واکنش سریع، هوشمند، مبتنی بر اینترنت اشیاء و کاملاً عملیاتی محسوب می‌گردد.

۶ پیوست ۱: کدهای نهایی (Source Code)

کدهای کامل، تاریخچه توسعه و مستندات این پروژه به صورت متن باز (Open-Source) در مخزن گیت هاب زیر قابل دسترسی می باشد:

GitHub Repository:

<https://github.com/NikaGhaderi/smart-security-esp32-uno>

۱.۶ کد کنترل پنل دسترسی (Arduino Uno)

این کد روی پردازنده ثانویه اجرا می شود، ورودی های کیپد را خوانده، رمز عبور را اعتبارسنجی کرده و فرمان مسلح سازی را صادر می کند.

```
#include <Keypad.h>

const byte ROWS = 4;
const byte COLS = 3;

char keys[ROWS][COLS] = {
  {'1','2','3'},
  {'4','5','6'},
  {'7','8','9'},
  {'*','0','#'}
};

byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {5, 4, 3};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

const String password = "789";
String currentInput = "";
const int commPin = 10;
bool isArmed = false;

void setup() {
  Serial.begin(9600);

  pinMode(commPin, OUTPUT);
  digitalWrite(commPin, LOW);

  Serial.println("Arduino Uno Access Panel Ready.");
  Serial.println("Enter PIN and press '5' to toggle the alarm.");
}

void loop() {
```

```

char key = keypad.getKey();

if (key) {
  if (key == '5') {
    Serial.println();

    if (currentInput == password) {
      isArmed = !isArmed;

      if (isArmed) {
        digitalWrite(commPin, HIGH);
        Serial.println("SUCCESS: System ARMED.");
      } else {
        digitalWrite(commPin, LOW);
        Serial.println("SUCCESS: System DISARMED.");
      }
    } else {
      Serial.println("ACCESS DENIED: Wrong PIN.");
    }
    currentInput = "";
  }
  else if (key == '*') {
    currentInput = "";
    Serial.println("\nInput Cleared.");
  }
  else {
    Serial.print("*");
    currentInput += key;
  }
}
}

```


۲.۶ کد هسته مرکزی، سنسورها و شبکه (ESP32-CAM)

این کد وظیفه مدیریت دوربین، سنسور PIR، هشدارهای محلی و ارتباط با کلودفلر برای ارسال عکس به تلگرام را بر عهده دارد.

```
#include "esp_camera.h"
#include <WiFi.h>
#include <WiFiClientSecure.h>

// --- 1. Network & Telegram Credentials ---
const char* ssid = "****";
const char* password = "****";
const String BOT_TOKEN = "****";
const String CHAT_ID = "****";
const String CF_WORKER = "****";

// --- 2. Camera Pin Definitions for AI-Thinker ---
#define PWDN_GPIO_NUM    32
#define RESET_GPIO_NUM   -1
#define XCLK_GPIO_NUM     0
#define SIOD_GPIO_NUM    26
#define SIOC_GPIO_NUM    27
#define Y9_GPIO_NUM      35
#define Y8_GPIO_NUM      34
#define Y7_GPIO_NUM      39
#define Y6_GPIO_NUM      36
#define Y5_GPIO_NUM      21
#define Y4_GPIO_NUM      19
#define Y3_GPIO_NUM      18
#define Y2_GPIO_NUM       5
#define VSYNC_GPIO_NUM   25
#define HREF_GPIO_NUM    23
#define PCLK_GPIO_NUM    22

// --- Custom Project Pins ---
const int pirPin = 13;      // The PIR Sensor
const int buzzerPin = 14;   // The local Siren
const int onboardLED = 33;  // The tiny red internal LED
const int normalLED = 4;    // External LED AND the massive white Flash LED
const int alertLED = 12;    // External Alert LED
const int commPin = 2;      // Armed/Disarmed Signal from Arduino Uno Pin 10

// --- 4. System States ---
bool hasCapturedPhoto = false;
```

```

void setup() {
    Serial.begin(115200);

    // Set up the Input/Output pins
    pinMode(pirPin, INPUT_PULLDOWN);
    pinMode(buzzerPin, OUTPUT);
    pinMode(onboardLED, OUTPUT);
    pinMode(normalLED, OUTPUT);
    pinMode(alertLED, OUTPUT);
    pinMode(commPin, INPUT_PULLDOWN);

    digitalWrite(buzzerPin, LOW);
    digitalWrite(onboardLED, HIGH);
    digitalWrite(normalLED, HIGH);
    digitalWrite(alertLED, LOW);

    // --- Connect to Wi-Fi ---
    delay(5000);
    Serial.print("Connecting to Wi-Fi");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWi-Fi connected!");

    // --- Configure Camera ---
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM;
    config.pin_href = HREF_GPIO_NUM;

```

```

config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;

// Resolution
if(psramFound()){
    config.frame_size = FRAMESIZE_SVGA;
    config.jpeg_quality = 10;
    config.fb_count = 2;
} else {
    config.frame_size = FRAMESIZE_VGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;
}

esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
}

Serial.println("System Ready! Waiting for Arduino Uno to ARM the system...");
}

void loop() {
    // Check if the Arduino Uno keypad has ARMED the system
    int systemArmed = digitalRead(commPin);

    if (systemArmed == HIGH) {
        int motionState = digitalRead(pirPin);

        if (motionState == HIGH) {
            // 1. MOTION DETECTED: Immediately trigger the physical deterrents!
            digitalWrite(buzzerPin, HIGH);
            digitalWrite(onboardLED, LOW);
            digitalWrite(normalLED, LOW);
            digitalWrite(alertLED, HIGH);

            // 2. Take the photo ONLY if we haven't already taken one for this movement
            if (!hasCapturedPhoto) {

```

```

        Serial.println("INTRUDER DETECTED! Sounding alarm and snapping photo...");
        captureAndSendPhoto();
        hasCapturedPhoto = true; // Lock the camera out until motion stops
    }

} else {
    // System is Armed, but no motion. Keep alarm quiet.
    digitalWrite(buzzerPin, LOW);
    digitalWrite(onboardLED, HIGH);
    digitalWrite(normalLED, HIGH);
    digitalWrite(alertLED, LOW);

    // Reset the photo flag so it is ready for the next intruder
    hasCapturedPhoto = false;
}

} else {
    digitalWrite(buzzerPin, LOW);
    digitalWrite(onboardLED, HIGH);
    digitalWrite(normalLED, HIGH);
    digitalWrite(alertLED, LOW);

    hasCapturedPhoto = false;
}

delay(100);
}

// --- Telegram Photo Transmission Function ---
void captureAndSendPhoto() {
    camera_fb_t * fb = NULL;

    // Flush the current frame to get a fresh image of the movement
    fb = esp_camera_fb_get();
    esp_camera_fb_return(fb);
    fb = esp_camera_fb_get();

    if(!fb) {
        Serial.println("Camera capture failed");
        return;
    }

    WiFiClientSecure client;
    client.setInsecure();

```

```

// Connect to the Cloudflare Worker instead of Telegram
Serial.println("Connecting to Cloudflare Bridge...");

if (client.connect(CF_WORKER.c_str(), 443)) {
    Serial.println("Connected! Sending image...");

    String head = "--SecurityBoundary\r\nContent-Disposition: form-data; name=\"chat_id\"; \r\n\r\n" +
    String tail = "\r\n--SecurityBoundary--\r\n";

    uint32_t imageLen = fb->len;
    uint32_t extraLen = head.length() + tail.length();
    uint32_t totalLen = imageLen + extraLen;

    // Send the HTTP POST headers to the Worker
    client.println("POST /bot" + BOT_TOKEN + "/sendPhoto HTTP/1.1");
    client.println("Host: " + CF_WORKER); // Update the Host header!
    client.println("Content-Length: " + String(totalLen));
    client.println("Content-Type: multipart/form-data; boundary=SecurityBoundary");
    client.println();

    // Send the body head
    client.print(head);

    // Send the image data in small chunks to prevent memory crashes
    uint8_t *fbBuf = fb->buf;
    size_t fbLen = fb->len;
    for (size_t n = 0; n < fbLen; n = n + 1024) {
        if (n + 1024 < fbLen) {
            client.write(fbBuf, 1024);
            fbBuf += 1024;
        } else if (fbLen % 1024 > 0) {
            size_t remainder = fbLen % 1024;
            client.write(fbBuf, remainder);
        }
    }

    // Send the body tail
    client.print(tail);

    // Wait for the response to clear the buffer
    while (client.connected() && client.available() == 0) {
        delay(10);
    }
}

```



```
}  
while (client.available()) {  
    client.read();  
}  
  
client.stop();  
Serial.println("Photo safely routed through Cloudflare to Telegram!");  
} else {  
    Serial.println("Failed to connect to Cloudflare server.");  
}  
  
// Free up the camera memory  
esp_camera_fb_return(fb);  
}
```

References

- [1] AI-Thinker Technology, *ESP32-CAM Audio/Video Camera Module Datasheet*, V1.2.
- [2] Espressif Systems, *ESP32 Series Datasheet*, 2023. Available: <https://www.espressif.com>
- [3] Arduino, *Arduino Uno Rev3 Hardware Reference*. Available: <https://docs.arduino.cc/hardware/uno-rev3>
- [4] Components101, *HC-SR501 PIR Motion Sensor Datasheet*. Available: <https://components101.com/sensors/hc-sr501-pir-sensor>
- [5] Telegram Messenger, *Telegram Bot API Documentation*. Available: <https://core.telegram.org/bots/api>
- [6] Cloudflare, *Cloudflare Workers Documentation*. Available: <https://developers.cloudflare.com/workers/>